

LangGraph Postgres Runtime

ops.py 重写报告 & Thread TTL 分析

2026-06-14

版本: langgraph_api 0.10.0 | langgraph_runtime_inmem | postgres_py

目录

1. 执行概述
2. 接口匹配原理
3. 返回类型变更汇总
4. 方法名变更汇总
5. Authenticated 基类实现
6. 嵌套类实现
7. Worker 内部方法
8. 代码质量审查与修复
9. 测试验证结果
10. Thread TTL 分析
11. Redis 数据清理机制
12. 待实现事项

1. 执行概述

本报告记录了 langgraph_runtime_postgres_py/ops.py 的完整重写过程，目标是匹配安装的 langgraph_api 0.10.0 所期望的 ops 接口（与 langgraph_runtime_inmem 模式一致）。

重写前，旧版 ops.py 使用简单的直接返回 dict/list 模式，与 langgraph_api 的消费方式（fetchone/get_pagination_headers）不兼容。重写后，所有方法签名、返回类型、Auth 参数均与 inmem 版本对齐。

核心变更统计

变更类别	数量	说明
方法重命名	5	create→put, update→patch, versions→get_versions
返回类型变更	15+	dict→AsyncIterator, list→tuple[AsyncIterator,int]
新增方法	8	count, set_latest, cancel, enter, Stream.*
新增嵌套类	3	Threads.State, Threads.Stream, Runs.Stream
Auth 集成	45+	每个公开方法添加 ctx 参数 + handle_event
代码质量修复	6	SQL注入、N+1查询、异常处理等

2. 接口匹配原理

langgraph_api 使用多态分发模式加载 ops 类：

```
if IS_POSTGRES_OR_GRPC_BACKEND:
    from langgraph_api.grpc.ops import Runs, Threads, Assistants, Crons
else:
    from langgraph_runtime.ops import Runs, Threads, Assistants, Crons
```

当 LANGGRAPH_RUNTIME_EDITION 不是 postgres/grpc 时，langgraph_runtime 会动态加载我们的后端包。因此我们的 ops.py 必须完全匹配 inmem 的接口规范。

核心消费模式

fetchone() - 单条结果消费

```
async def fetchone(it, not_found_code=404):
    try:
        return await anext(it) # ■ AsyncIterator ■■■ yield
    except StopAsyncIteration:
        raise HTTPException(status_code=not_found_code)
```

所有 put/get/patch/delete 方法必须返回 AsyncIterator，让 fetchone 消费。空迭代器触发 404。

get_pagination_headers() - 分页结果消费

```
async def get_pagination_headers(resource, next_offset, offset):
    resources = [r async for r in resource] # ■■■ list
    if next_offset is None:
        headers = {'X-Pagination-Total': str(len(resources) + offset)}
    else:
        headers = {'X-Pagination-Next': str(next_offset), ...}
    return resources, headers
```

search() 方法必须返回 tuple[AsyncIterator, int], 其中 int 是下一页 offset 或 None。实现方式: 查询 LIMIT limit+1, 若返回 limit+1 行则有更多页, 否则 cursor=None。

3. 返回类型变更汇总

下表列出所有方法的旧返回类型→新返回类型及消费方式:

方法	旧返回类型	新返回类型	消费方式
Assistants.put	dict	AsyncIterator[dict]	fetchone(not_found_code=409)
Assistants.get	dict None	AsyncIterator[dict]	fetchone()
Assistants.search	list[dict]	tuple[AsyncIterator, int None]	get_pagination_headers()
Assistants.patch	dict	AsyncIterator[dict]	fetchone()
Assistants.delete	None	AsyncIterator[UUID]	fetchone() (drain)
Assistants.count	-	int	直接返回
Assistants.set_latest	-	AsyncIterator[dict]	fetchone()
Assistants.get_versions	list[dict]	AsyncIterator[dict]	async for 迭代
Threads.put	dict	AsyncIterator[dict]	fetchone(not_found_code=409)
Threads.get	dict None	AsyncIterator[dict]	fetchone()
Threads.search	list[dict]	tuple[AsyncIterator, int None]	get_pagination_headers()
Threads.patch	dict	AsyncIterator[dict]	fetchone()
Threads.delete	None	AsyncIterator[UUID]	fetchone() (drain)
Threads.count	-	int	直接返回
Threads.State.get	Any	StateSnapshot	直接返回 (非迭代器)
Threads.State.post	dict	ThreadUpdateResponse	直接返回 (非迭代器)
Threads.State.list	list[Any]	list[StateSnapshot]	直接返回 list
Runs.put	dict	AsyncIterator[dict]	anext() + async for
Runs.get	dict None	AsyncIterator[dict]	fetchone()
Runs.search	list[dict]	AsyncIterator[dict]	async for 迭代 (无分页)
Runs.delete	None	AsyncIterator[UUID]	fetchone() (drain)
Runs.cancel	-	None	直接返回
Runs.enter	-	AsyncIterator[ValueEvent]	@asynccontextmanager
Crons.put	dict	AsyncIterator[dict]	fetchone()
Crons.search	list[dict]	tuple[AsyncIterator, int None]	get_pagination_headers()
Crons.delete	None	AsyncIterator[UUID]	fetchone() (drain)
Crons.count	-	int	直接返回

4. 方法名变更汇总

类	旧方法名	新方法名	备注
Assistants	create()	put()	增加 assistant_id, if_exists, system 参数
Assistants	update()	patch()	增加 ctx, config/context 互补逻辑
Assistants	versions()	get_versions()	增加 metadata 过滤, ctx
Threads	create()	put()	增加 thread_id, if_exists, ttl 参数
Threads	update()	patch()	增加 ttl, read_mask_paths
Runs	create()	put()	增加 大量新参数 (multitask 等)
Runs	update()	set_status()	简化为 worker 内部方法, 返回 None
Crons	create()	put()	增加 cron_id, enabled, timezone 参数

5. Authenticated 基类实现

每个 ops 类继承 Authenticated, 提供统一的认证上下文处理:

```
class Authenticated:
    resource: Literal['threads', 'crons', 'assistants']

    @classmethod
    def _context(cls, ctx, action) -> Auth.types.AuthContext | None:
        if not ctx: return None
        return Auth.types.AuthContext(
            user=ctx.user, permissions=ctx.permissions,
            resource=cls.resource, action=action)

    @classmethod
    async def handle_event(cls, ctx, action, value):
        from langgraph_api.auth.custom import handle_event
        from langgraph_api.utils import get_auth_ctx
        ctx = ctx or get_auth_ctx()
        if not ctx: return None
        return await handle_event(cls._context(ctx, action), value)
```

每个公开方法调用 handle_event 获取过滤条件, 然后用 _check_filter_match(metadata, filters) 检查权限。

6. 嵌套类实现

Threads.State

线程状态操作, 直接返回对象 (非 AsyncIterator) :

方法	返回类型	说明
get(conn, config, subgraphs)	StateSnapshot	获取线程当前状态快照
post(conn, config, values, as_node)	ThreadUpdateResponse	更新线程状态
bulk(conn, config, supersteps)	ThreadUpdateResponse	批量更新状态

方法	返回类型	说明
list(conn, config, limit, before)	list[StateSnapshot]	获取状态历史

Threads.Stream

线程流操作，用于 SSE 事件推送：

方法	返回类型	说明
join(thread_id, ...)	AsyncIterator[tuple[bytes,bytes,bytes None]]	SSE EventSourceResponse
publish(thread_id, event, message)	None	发布线程流事件
subscribe(thread_id)	ContextQueue	订阅线程流
check_thread_stream_auth(thread_id)	None	权限检查

Runs.Stream

Run 流操作，用于 SSE 事件推送：

方法	返回类型	说明
subscribe(run_id, thread_id)	ContextQueue	订阅 run 流
join(run_id, stream_channel, ...)	AsyncIterator[tuple[bytes,bytes,bytes None]]	SSE 流输出
check_run_stream_auth(run_id, thread_id)	None	权限检查
publish(run_id, event, message)	None	发布 run 事件

7. Worker 内部方法

这些方法不走 HTTP API，由 worker 进程直接调用：

方法	返回类型	调用位置	作用
Threads.set_status(conn, tid, cp, exc)	None	worker.py	更新线程状态
Threads.set_joint_status(conn, tid, rid, ...)	None	worker.py	原子更新线程+运行状态
Runs.set_status(conn, rid, status)	None	worker.py	更新 run 状态
Runs.cancel(conn, run_ids, action, ...)	None	worker/api	取消 run
Runs.enter(rid, tid, loop, resumable)	ValueEvent	worker.py	@asynccontextmanager 进入 run 执行上下文
Runs.next(wait, limit)	AsyncIterator[tuple[R un,int]]	worker.py	从队列取 run

方法	返回类型	调用位置	作用
Runs.sweep()	None	定时任务	清理超时 run

8. 代码质量审查与修复

代码质量审查发现 4 个严重问题和 6 个重要问题，均已修复：

严重问题（已修复）

编号	问题	修复方案
C1	Runs.sweep() 中 INTERVAL 使用 f-string 拼接，SQL 注入风险	改为参数化查询 INTERVAL '%s seconds'
C2	Runs.cancel() 被以 conn=None 调用会崩溃	添加 AsyncExitStack 模式获取连接
C3	Runs.Stream.subscribe() 创建孤立 pubsub，队列永远为空	改用 StreamManager.add_queue()
C4	Threads.Stream.subscribe() 同上	改用 StreamManager 正确连接

重要问题（已修复）

编号	问题	修复方案
I1	Runs.cancel() 中 N+1 查询获取线程元数据	批量 WHERE thread_id IN (...) 查询
I2	Runs.search() 中每行都查询线程元数据	循环外一次查询
I4	缺失记录错误处理不一致	统一 patch/delete 抛 404
I5	Threads.delete() 中裸 except Exception:pass	添加 logger.warning()
I6	Runs.delete() 同上	添加 logger.warning()
I7	count() 方法在有 auth filter 时加载全部行	待优化，目前可接受

9. 测试验证结果

Demo 测试

Demo	内容	结果
Demo 1	Counter Graph (increment → double 循环)	☐ 通过
Demo 2	Human-in-the-Loop (中断后恢复)	☐ 通过
Demo 3	完整 API 流程 (Assistant→Thread→Run→Events)	☐ 通过

PostgreSQL 数据验证

表名	记录数	说明
assistants	2	注册的 counter + agent 图
assistant_versions	2	每个图一个版本
threads	0	Demo 清理后删除
runs	0	Demo 清理后删除
checkpoints	59	完整的 checkpoint 链
checkpoint_writes	122	checkpoint 写入操作

Checkpoint 链验证: parent_checkpoint_id 关系正确建立, 从根节点 (parent=None) 到叶子节点形成完整的状态历史链。

服务器启动测试

通过 start_server.py 启动 langgraph_api 服务, 验证:

- /ok 端点返回 {"ok": true}
- /assistants/search 返回已注册的图列表
- Graph 注册成功 (counter + agent)
- 服务器无 AttributeError 或接口不匹配错误

10. Thread TTL 分析

10.1 ThreadTTLConfig 定义

ThreadTTLConfig 是线程生命周期管理配置，包含三个字段：

字段	类型	说明
strategy	Literal['delete', 'keep_latest']	'delete': 删除线程及全部数据；'keep_latest': 保留最新状态，清理旧 checkpoint
default_ttl	float None	线程存活时间（分钟），超期后执行 strategy
sweep_interval_minutes	int None	扫描间隔（分钟），定时检查过期线程

10.2 TTL 两种策略

strategy='delete'

线程过期时：删除 thread 行 + 所有 runs + 所有 crons + 所有 checkpoint (PG saver 数据)。这是彻底清理，释放所有磁盘空间。

strategy='keep_latest'

线程过期时：只清理旧的 checkpoint 历史，保留线程和最新状态。注意：inmem 后端不支持此策略，会抛 HTTPException(422)。

10.3 TTL 执行机制对比

后端	TTL 清扫	机制	状态
grpc (Go)	□ Go 后端内部定时清扫	Python 发送 sweep_interval → Go 自己跑定时任务	已实现
inmem	□ 空壳	sweep_ttl() 直接 return (0, 0)	未实现
postgres_py	□ 未实现	需要自己实现	待实现

10.4 TTL 清扫删除的数据范围

当线程因 TTL 过期被清理时，以下数据被删除：

数据位置	是否删除	说明
threads 表行	□	strategy='delete' 时删除；'keep_latest' 时保留
runs 表行	□	级联删除该线程的所有 run
crons 表行	□	级联删除该线程的所有 cron
checkpoints (PG saver)	□	调用 checkpointer.delete_thread(thread_id)
checkpoint_writes	□	随 checkpoint 级联删除
Redis Stream 数据	□	依赖自身 TTL (120秒) 自然过期

10.5 配置方式

全局 TTL 配置通过环境变量：

```
LANGGRAPH_THREAD_TTL={"strategy":"delete","default_ttl":60,"sweep_interval_minutes":5}
```

单线程 TTL 通过 API 设置：

```
POST /threads {"ttl": {"strategy": "delete", "ttl": 30}} # 30■■■■■
```

实际存储：计算得到 `expires_at = now + timedelta(minutes=ttl)`，写入 `threads.expires_at` 列。

11. Redis 数据清理机制

11.1 Redis 中存储的数据类型

数据类型	Redis Key 模式	TTL
Run 事件流	<code>run:{run_id}:events</code>	<code>RESUMABLE_STREAM_TTL_SECONDS</code> (120秒)
Run 模式流	<code>run:{run_id}:modes</code>	同上
任务队列	<code>langgraph:runs</code>	无 TTL，消费后删除
Worker 心跳	<code>langgraph:workers</code>	心跳超时自动过期

11.2 线程删除时 Redis 的处理

关键发现：Thread 删除时，Redis stream 数据没有被显式清理。

- 无论是 inmem 还是 grpc 后端，`Threads.delete()` 都不包含 Redis 清理代码
- Redis Stream 数据依赖自身 TTL 机制（默认 120 秒）自然过期
- 这意味着线程删除后，最多 2 分钟内仍有可能通过 stream 读到该线程的残留事件
- 对于生产环境，这是可接受的，因为 stream 消费者会检查 run 状态并自行终止

11.3 对 postgres_py 的建议

在我们的 postgres_py 后端中，Redis 清理策略：

- 保持现状：依赖 Redis Stream 自身 TTL (120秒) 自然过期，不做显式删除
- 如需更快清理，可在 `Threads.delete()` 中添加 Redis `XTRIM/XDEL` 操作
- 但考虑到 Redis stream key 是按 `run_id` 而非 `thread_id` 组织的，需要先查询该线程的所有 `run_id` 才能清理

12. 待实现事项

优先级	事项	说明
P0	实现 <code>Threads.sweep_ttl()</code>	扫描 <code>expires_at < NOW()</code> 的线程并删除，返回 (<code>deleted_count</code> , <code>checkpoint_count</code>)
P0	在 <code>lifespan</code> 中启动 TTL 清扫定时任务	每 <code>sweep_interval_minutes</code> 分钟调用一次 <code>sweep_ttl()</code>
P1	实现 <code>keep_latest</code> 策略	只清理旧 <code>checkpoint</code> ，保留线程和最新状态
P1	优化 <code>count()</code> 方法的 <code>auth filter</code>	当前在有 <code>auth filter</code> 时加载全部行计数，应推入 SQL

优先级	事项	说明
P2	添加 Redis 显式清理（可选）	在 Threads.delete() 中清理该线程的 run stream
P2	Runs.next() 轮询优化	添加指数退避策略，减少空转时的资源浪费

— 报告结束 —